

TP POO PHP

Gestion des Utilisateurs

Université / Institut

Objectif

Ce TP a pour objectif de mettre en pratique les concepts fondamentaux de la programmation orientée objet en PHP :

- Classes et objets
- Encapsulation
- Héritage
- Redéfinition de méthodes
- Classes abstraites
- Interfaces
- Polymorphisme
- Attributs et méthodes statiques
- Constantes

Contexte

On souhaite développer un système de gestion des utilisateurs d'une application. Il existe plusieurs types d'utilisateurs :

- Client
- Employé
- Administrateur

Chaque utilisateur possède des informations communes mais aussi des comportements spécifiques.

Travail demandé

Partie 1 : Classe de base

Créer une classe **Personne** contenant :

- Attributs privés : id, nom, email
- Un constructeur
- Les getters et setters
- Une méthode `afficherInfos()`

Partie 2 : Héritage

Créer une classe **Utilisateur** qui hérite de **Personne**.

Ajouter :

- login
- motDePasse
- Méthode `seConnecter()`

Partie 3 : Classes filles

Créer les classes suivantes :

- **Client**
 - Attribut : `typeClient` (simple ou premium)
 - Méthode `calculerReduction($montant)`
- **Employe**
 - Attribut : `salaire`
 - Méthode `calculerSalaireAnnuel()`
- **Administrateur**
 - Méthode `supprimerUtilisateur()`

Partie 4 : Redéfinition

Dans la classe **Client**, redéfinir la méthode `afficherInfos()` afin d'afficher également le type de client.

Utiliser le mot-clé `parent`.

Partie 5 : Classe abstraite

Transformer la classe **Utilisateur** en classe abstraite.

Ajouter la méthode abstraite :

```
abstract public function afficherRole();
```

Implémenter cette méthode dans chaque classe fille.

Partie 6 : Interface Authentifiable

Créer une interface **Authentifiable** :

```
interface Authentifiable {  
    public function seConnecter();  
}
```

Faire implémenter cette interface par la classe **Utilisateur**.

Partie 7 : Interface Affichable

Créer une interface **Affichable** :

```
interface Affichable {  
    public function afficher();  
}
```

Faire implémenter cette interface par vos classes.

Partie 8 : Polymorphisme

Créer la fonction suivante :

```
function afficherUtilisateur(Affichable $u) {  
    $u->afficher();  
}
```

Tester avec différents objets.

Partie 9 : Statique

Dans la classe **Utilisateur** :

- Ajouter un attribut statique `nombreUtilisateurs`
- L'incrémenter dans le constructeur
- Créer une méthode statique `afficherNombre()`

Partie 10 : Constantes

Dans la classe **Client**, définir :

```
const TAUX_SIMPLE = 0.05;  
const TAUX_PREMIUM = 0.15;
```

Utiliser ces constantes dans le calcul de réduction.

Tests

Dans le programme principal :

- Créer un client
- Créer un employé
- Créer un administrateur
- Tester les méthodes :
 - `afficher`
 - `seConnecter`
 - `calculerReduction`
 - `calculerSalaireAnnuel`
 - `polymorphisme`